How a payment actually executes

The platform design, consolidated from seven revisions into one. The resource model, the control plane, the execution graph, the credential boundary, money movement and the migration — with the three count discrepancies and the absent concepts named rather than smoothed over.

Document	PD-001 · v1.0
Date	27 September 2026
Consolidates	Platform specification v3.6 (34 sections)
Companion	PD-002 · decisioning and evidence
Implements	EA-001 v1.1 · EA-002 v1.0
Status	Canonical — supersedes six revisions

Contents

PART I — FOUNDATIONS

01 How to read this document

02 What holds the design together

03 Five nested resources

04 The transaction context

PART II — CONTROL AND EXECUTION

05 The control plane

06 The execution plan

07 Attempt strategy

08 Rail execution

09 Recovery

PART III — CREDENTIAL, SECURITY AND MONEY

10 Credential and instrument

11 Platform security

12 Money movement

13 Cross-border, FX and multi-currency

PART IV — RECORDS, INTERFACES AND GETTING THERE

14 State machines

15 Journal, determinism and replay

16 Canonical results and disclosure

17 The API surface

18 Latency and data placement

19 Migration and legacy coexistence

20 What this document does not answer

PART

I — Foundations

01 How to read this document

02 What holds the design together

03 Five nested resources

04 The transaction context

01

How to read this document

> This is the platform specification, consolidated. It supersedes six earlier revisions of the same document, which are retired to the shelf described in DOC-000 section 03.

THIS DOCUMENT COVERS	THIS DOCUMENT DOES NOT COVER
The resource model, transaction context, control plane, execution model, rail execution, attempt strategy, recovery, credential and instrument, platform security, screening, authentication, money movement, currency, state machines, the journal, the error model, observability, the API surface, latency, data placement and migration.	How a decision is made, tested or proven. Decision points, strategies, flows, routing, policy, experimentation and measurement are all in **PD-002**. This document describes the machine; the companion describes the judgement.

THREE THINGS THAT ARE NOT IN THE SOURCE, AND ARE NOT INVENTED HERE

Consolidating surfaced four concepts that circulate in conversation and appear nowhere in the specification. They are named here so that nobody assumes they were dropped in consolidation:

- **PAR** — the payment account reference is never mentioned. It is the only cross-channel link between a wallet token and its funding account, and its absence is a genuine gap for the omnichannel case.
- **Vaultless tokenisation and format-preserving encryption** — not present. The vault here is stateful throughout.
- **Key epochs** — key rotation is specified; epoch vocabulary is not. The nearest concept is the ownership epoch, which is a migration construct, not a cryptographic one.

Each is new material if wanted, not a restatement. They are added to the open register in DOC-000 rather than quietly written in here. [A]

Three count discrepancies, carried forward rather than corrected

IN THE SOURCE	WHAT IS ACTUALLY THERE	WHAT THIS DOCUMENT DOES
“The six binding constraints”	Seven are listed.	Lists seven, calls them seven.
“The ten externally monetisable domains”	Twelve rows are marked sellable.	Leaves the table as the authority and drops the prose count.
“Twenty-eight owned domains” in the open questions, against thirty-three bounded contexts in the domain inventory	Two different numbers for what reads as one thing.	Keeps thirty-three. The organisational count is a different property and belongs to ST-001.

The open-question list is also numbered out of sequence in the source — one through thirteen, then seventeen and eighteen, then fourteen to sixteen — which is an append artefact from the later revisions. Section 20 here renumbers it, and the cross-references that cited items by number are updated with it.

02

What holds the design together

> A payment is an immutable business intent. An execution plan determines which versioned capabilities run, in what order, under which constraints — with every decision logged with enough context to be replayed, attributed and second-guessed.

The four separations

CONCERN	THE QUESTION IT ANSWERS	CHANGES WHEN	OWNER
Decisioning	What should happen to this transaction?	Policy, model or strategy changes	Product, risk, compliance
Orchestration	In what order do capabilities run, and what happens when one fails?	Plan version changes	Platform engineering
Capability execution	How is one decision computed or one action performed?	Module version changes	The capability team
State management	What is true about this payment now, and what was true before?	Almost never	Platform engineering

The nine principles

#	PRINCIPLE	WHAT IT RULES OUT
1	**Intent is immutable; execution is versioned.**	Editing what a payment meant in order to change how it was tried.
2	**Every decision carries provenance.**	A decision nobody can attribute to a decider, a version, an input set and a regime.
3	**Constraints are hierarchical and non-negotiable downward.**	Optimization overriding regulation, at any level, for any reason.
4	**Learning is bounded by entitlement.**	A platform that only works well for merchants who granted autonomy.
5	**Precompute over lookup.**	Any remote lookup on the authorization path for something resolvable at vault time or config-push time.
6	**Recovery is a domain, not an afterthought.**	An unknown-outcome timeout with no owner, which is the default state of most payment platforms.
7	**Internal state vocabulary is large; external is small.**	Exporting our complexity to merchants as a status enum they have to handle.
8	**Disclosure is a function of role and delegation, not a table of booleans.**	A static access matrix that cannot express what a PayFac may see about its own submerchant.
9	**Degrade explicitly.**	A dependency failure resolving to an incident rather than to a declared behaviour.

The module contract

```
module(input: TransactionContext + explicit dependencies) → Decision | Effect | Failure
```

PROPERTY	MEANING
stateless	No mutation of payment state.
versioned	Semver, deployable independently.
deterministic	Same input and same dependency versions produce the same output — models included, because the artefact version is an input.
idempotent	For any module producing effects.
timeout-controlled	A hard deadline, declared rather than discovered.
observable	Emits a module execution record always, including on timeout and on failure.
fallback-declared	States its own degraded behaviour. The orchestrator does not invent one.

THE ANTI-PATTERN THE CONTRACT EXISTS TO PREVENT

A routing module that reads the database, mutates the payment, calls the processor and publishes events. Each of those is reasonable on its own; together they recreate the monolith inside a module boundary, and the boundary then makes it harder to see rather than easier.

The seven binding constraints

The source calls these six and lists seven. They are the conditions the design has to survive, not aspirations: we are a bank rather than a fintech; we are late; our volume advantage is real but concentrated; migration is not greenfield; enterprise merchants demand determinism; the unit of competition is not the gateway; and simplicity is a requirement rather than a preference.

Four things are deliberately not goals: exactly-once processing, a universal ledger for the enterprise, real-time model training, and full processor abstraction on day one.

Five nested resources

FIVE NESTED RESOURCES · AND WHAT EACH ONE EXISTS TO EXPRESS

OBLIGATION
what is owed, by whom, to whom, under what terms
the only object with a counterparty and a due date; survives its order

ORDER
a coordinated collection effort against obligations
may span payers, instruments, currencies and timing

PAYMENT INTENT
one payer's commitment to pay a portion
carries the mandate and the authentication context, so both survive retries

PAYMENT REQUEST
one instruction to move money
binds the plan version; authorize, capture, refund, reversal

ATTEMPT
one execution of that instruction
one processor, one credential — the richest object in the system

A STANDARD CHECKOUT CREATES ALL FIVE. THE DEVELOPER SEES ONE.

Implicit objects are real objects — addressable, retrievable, identical in reporting — not hidden state.

— Obligation exists because a franchise royalty, a marketplace split, a dunning sequence and a B2B invoice with partial payment are all inexpressible without it. Progressive disclosure is why that costs the ordinary developer nothing.

RESOURCE	KEY FIELDS	STATES
Obligation	`obligation_id` · obligor · obligee · original and outstanding amount · basis (sale, subscription, royalty, fee, settlement, adjustment, penalty, refund liability) · schedule · discharge policy · `linked_orders[]` · `ledger_account_ref`	`OPEN · PARTIALLY_DISCHARGED · DISCHARGED · DISPUTED · WRITTEN_OFF · CANCELLED`
Order	A coordinated collection effort. May span payers, instruments, currencies and timing.	`CREATED → COLLECTING → PARTIALLY_COLLECTED → FULLY_COLLECTED → DISCHARGING → SETTLED`, with terminal `FAILED | EXPIRED | CANCELED`
Payment intent	`intent_id` · `obligation_refs[]` · payer · authorized ceiling · `instrument_ref` (a vault reference, never a credential) · mandate · **authentication context** · capture method · `industry_context_ref`	Ten external states — see section 14
Payment request	`request_id` · operation · amount · `idempotency_key` · `exec_id` · `plan_version` (bound at resolution) · `attempts[]`	Operations: `AUTHORIZE · INCREMENTAL · CAPTURE · PARTIAL_CAPTURE · VOID · REFUND · REVERSAL · TRANSFER_IN · TRANSFER_OUT`
Attempt	The richest object in the system: sequence · `exec_id` · plan version · every decision with its version, reason code, regime and latency · strategy origin · processor, acquirer, network, merchant account · message refs · canonical result · timings · `recovery_ref`	Outcome: `APPROVED | DECLINED | ERROR | TIMEOUT | UNKNOWN`. Origin: `INITIAL | STEP_UP_AUTH | CREDENTIAL_SUB | PROCESSOR_SUB | NETWORK_SUB | SCHEDULED_RETRY`

WHY OBLIGATION EXISTS, AND WHY IT COSTS THE ORDINARY DEVELOPER NOTHING

A franchise royalty, a marketplace split that outlives its collection, subscription dunning, instalments, a deposit then a balance, and a B2B invoice with partial payment are all inexpressible in a model that stops at Order. A collection wrapper cannot express an obligation that outlives the collection.

Progressive disclosure is the answer to the cost. A standard checkout posts a payment intent with amount, currency, payment method and capture method; the platform implicitly creates the obligation, the order, the request and the attempt. **Implicit objects are real objects** — addressable, retrievable, identical in reporting — not hidden state. Explicit construction is required only for multiple payers, splits that outlive the collection, schedules, or obligations between parties other than customer and merchant.

This is a binding constraint rather than an SDK convenience, because a developer-experience objection answered only in the SDK gets re-litigated at every API review. [A]

The transaction context

> The context is assembled once, immutable thereafter, and is the sole input to plan resolution and every module. If a module needs something not in the context, either the context is wrong or the module is reaching outside its contract.

BLOCK	CONTENTS
Hierarchy	Eleven levels: psp, legal entity, account, platform, payfac, isv, merchant, submerchant, location, terminal, customer
Commercial	Obligation basis, order shape, industry, MCC, pricing plan, contract terms
Geography	Merchant, acquiring, issuer, billing, shipping and IP country; data residency zone
Monetary	Amount, presentment and settlement currency, FX treatment, amount band
Instrument	Method, network, funding type, product id, commercial, prepaid, regulated debit, HSA/FSA, token type, token requestor, wallet, credential age, updater status — all resolved at vault time
Channel	Card present or not, ecom, MOTO, in-app, terminal, QR, unattended, entry mode, PIN capability
Mandate	CIT/MIT, recurring, instalment, credential on file, network transaction id, agreement reference
Constraint	The resolved constraint set, including SCA applicability and Reg II posture
Operation	The requested operation and its parameters
Experiment	The assignment vector — see PD-002 for where in the flow it is set, which is not here

THE ORDERING CORRECTION THAT MATTERS MOST

Instrument attributes resolve **before** constraint evaluation, not after. A reviewed design put policy at step three and BIN resolution at step four, which makes the most common constraint class — debit-scoped, regulated-debit-scoped, product-scoped — impossible to evaluate at all.

In this design the question does not arise, because the attributes arrive ***with the credential*** from vault-time enrichment. There is no runtime lookup to order. [A]

The effective configuration snapshot

Eleven levels of hierarchy are not traversed per transaction. They are resolved ahead of time into one signed immutable snapshot carrying capability, entitlement, preference and policy, plus routing, pricing and risk posture, the active strategies and their populations, and the available experiment layer assignments. It is recomputed on any contributing change, pushed like a plan bundle, and consumed as a single in-memory read.

The hierarchy is a control-plane concern; the data plane sees only its result.

PART

II — Control and execution

05 The control plane

06 The execution plan

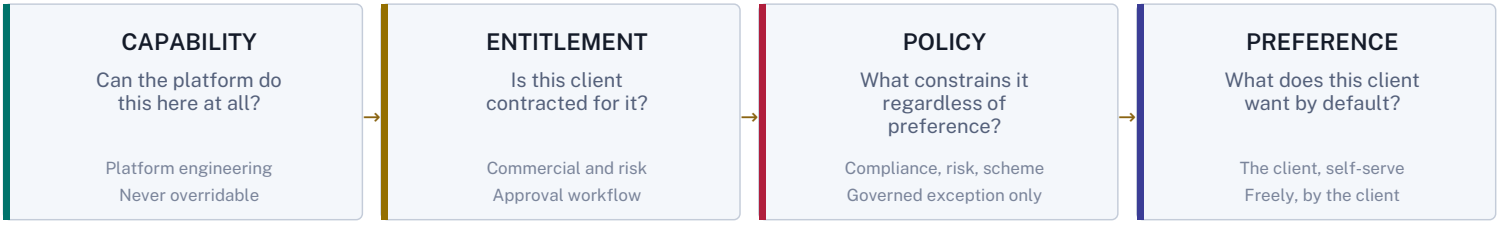
07 Attempt strategy

08 Rail execution

09 Recovery

The control plane

FOUR PLANES · RESOLVED IN ORDER, NEVER MERGED



ACTION SPACE = capability ∩ entitlement ∩ policy, ordered by preference

A strategy selects within it. A strategy cannot expand it — and that is checkable at compile time.

— A preference that exceeds an entitlement is not an error. It is inert, and it is reported as inert to whoever set it. Silently ignoring it is the behaviour this design exists to prevent.

The authority ladder

LEVEL	AUTHORITY	WHAT IT SETS
1	**Regulator**	SCA, Reg II, sanctions, residency, limits
2	**Network / scheme**	Scheme rules, mandates, prohibited practice
3	**PSP institutional**	Risk appetite, prohibited MCC, exposure limits
4	**Platform**	Platform operator rules over its estate
5	**PayFac**	PayFac rules over its submerchants
6	**Merchant**	Merchant configuration
7	**Submerchant**	Submerchant configuration
8	**Learned strategy**	Optimization, operating strictly inside one to seven

LEVEL EIGHT IS THE WHOLE POINT OF THE LADDER

Optimization sits below every human and regulatory authority. It chooses among options that levels one to seven have already declared acceptable, and **it never expands the option set**.

Higher levels bind lower ones unless authority is explicitly delegated downward with a recorded delegation grant, carrying scope, bounds and an expiry. Delegation is the only mechanism by which authority moves down.

Bitemporality, and the three questions it makes answerable

Every control-plane record carries two time dimensions: `valid_from` and `valid_to` for when the rule applies in the world, and `recorded_at` and `superseded_at` for when we learned it.

QUESTION	ANSWERED BY
What rule was in force when this transaction ran?	Valid time at the transaction timestamp.
What did we *believe* was in force when we ran it?	Transaction time at the transaction timestamp. If the two differ we misapplied a rule, and can quantify the exposure.
Which transactions were decided under a rule we later corrected?	The retroactive-correction query. This is the question in a regulatory finding, and without knowledge time it is unanswerable.

Two propagation channels

	NORMAL PATH	EMERGENCY PATH
Mechanism	compile → sign → push → adopt	signed constraint delta, direct to edge
SLO	60 s p99 globally	**5 s p99 globally**
Scope	Everything	**PROHIBIT and REQUIRE only** — never preferences, never plan changes, never routing or model artefacts
Authority	Any	Regulator, network or PSP levels only
Approval	Per the change tier	**Dual control — two approvers, both logged**
Direction	Either	Always additive. May restrict, never permit.
Expiry	Per the artefact TTL	30 days, with a mandatory post-review

FAIL-CLOSED, AND WHY THE EMERGENCY CHANNEL HAD TO EXIST

A node that cannot confirm it has applied an emergency delta within its deadline stops accepting traffic for the affected scope. Every pushed artefact carries a TTL; on expiry a node degrades to the most restrictive constraint set it holds and reports degraded. There is no configuration in which a node evaluates indefinitely on an artefact it cannot refresh.

The channel was not proposed by any review. Reviews framed plan distribution as a caching problem, and caching does not answer how a sanctions addition reaches every edge node in seconds. [A]

The execution plan

> The execution identifier is the platform's central abstraction: an opaque handle to a compiled plan, not an encoded string.

A plan carries its `exec_id`, its `plan_version`, a profile, what it was compiled from (capability set, entitlement snapshot, policy set, preference snapshot), the graph, the compensation block and a signature.

The graph, with real deadlines

NODE	VERSION	DEADLINE	DEPENDS ON	ON FAILURE
`screening`	4.1	12 ms	—	**HARD_FAIL**
`constraint_bind`	12.1	2 ms	—	**HARD_FAIL**
`risk`	19.4	18 ms	constraint_bind	FALLBACK
`authentication`	31.0	6 ms	risk, constraint_bind	FALLBACK
`credential_select`	8.3	3 ms	constraint_bind	FALLBACK
`routing`	23.1	4 ms	risk, credential_select	FALLBACK
`message_enrich`	11.7	3 ms	routing, authentication	DEGRADE
`authorize`	18.2	8000 ms	message_enrich	RECOVER

Screening, constraint binding and credential selection run concurrently at t=0. Risk gates authentication because authentication consumes the score. Routing consumes both risk and the selected credential. The critical path is what the latency budget in section 18 measures, not the sum.

SCREENING IS THE ONLY MODULE WITH NO FALLBACK

If screening cannot reach a decision within budget, the transaction fails. Every other module degrades; this one does not, and the `HARD_FAIL` disposition exists specifically for it.

Any future module requesting HARD_FAIL requires architecture review. The disposition is a privilege, not a configuration value.

What is versioned

ARTEFACT	VERSIONING	EXPOSURE
Plans	`plan_version`	By predicate over context dimensions, not only by percentage. This is the macro-experiment mechanism: a version can be 87% stable, 12% general ramp, 1% canary excluding strategic accounts and large amounts, and 0% internal dogfood.
Modules	Semver, per node	Through the plan
Artefacts	Policy set, routing table, BIN snapshot, cost table, risk model, route model, entitlement and preference snapshots, capability set, enrichment and feed versions	Pushed
Strategies	`strategy_id` and `strategy_version`	See PD-002
Message formatting	Per issuer or BIN range, by network, by channel	Its own experiment layer
Interchange tables	Versioned and bitemporal	Consumed identically by routing and billing

Plans compile centrally and distribute as signed, versioned, TTL-stamped bundles through a replicated regional registry to an in-process LRU on the edge node, with the signature verified on load. A rollback pointer makes version selection a pointer change rather than a redeploy. The cache keeps a negative entry too — "no plan for this context shape" is itself cached — coalesces concurrent fetches per key per node, and pre-warms a new version before its exposure predicate is switched on.

The module execution record

Emitted for every node on every attempt, including timeouts. One artefact, three consumers: operations, learning and audit.

```
attempt_id · node · version · status (completed | fallback | timeout | error)
started_at · duration_ms · input_digest
decision { processor, reason_code, regime, considered[], propensity }
dependencies { routing_table, bin_snapshot, model, policy_set }
experiment { cell, assignment_id }

regime ∈ POLICY | HEURISTIC | LEARNED | FALLBACK
propensity is required whenever regime = LEARNED
```

07

Attempt strategy

> Evaluates once, produces one next action. The interesting part is not the seven actions; it is the three independent ways it stops.

ACTION	WHEN
`STOP`	Hard decline, fraud, policy block, stop payment, or budget exhausted
`STEP_UP_AUTHENTICATION`	Soft decline where the issuer signals authentication would clear it. 3DS as a recovery action, carrying the original intent forward
`SUBSTITUTE_CREDENTIAL`	Network token to PAN or back, an updated credential, an alternate stored credential
`SUBSTITUTE_NETWORK`	An alternate debit network, where the constraint set permits
`SUBSTITUTE_PROCESSOR`	An alternate processor for availability or acceptance reasons
`ENRICH_AND_RETRY`	Same route, richer message
`SCHEDULE_RETRY`	A deferred attempt at a computed time, for issuer-state declines

Three independent termination mechanisms

None of them relies on the others being correct, which is the design requirement.

MECHANISM	HOW IT WORKS
Monotonic progress	A substitution vector over credential, network, processor, authentication state, message enrichment and amount. A new attempt must differ from every prior attempt on this request in at least one dimension, and the visited set is part of the request's state.
Typed budgets	Per dimension, not one counter: step-ups ≤ 1 per intent, credential substitutions ≤ 2, network substitutions ≤ 1, processor substitutions ≤ 2, scheduled retries per scheme rule, and a total ceiling that always binds lowest.
Hard ceiling and escape	An absolute per-request attempt ceiling and a wall-clock deadline, both enforced by the orchestrator rather than the strategy module. On breach: a canonical failure to the merchant and a supervisor case to recovery.

`WHAT IS NEVER PERMITTED, AND ONE CASE THAT LOOKS LIKE A RETRY AND IS NOT`

Never: blind repetition of an identical failed attempt; retry after a hard decline, a fraud decline or a stop payment; step-up where the constraint set prohibits authentication; substitution that would change the cardholder's agreement.

And the case that catches people: a substitution that alters SCA applicability, interchange treatment or acquiring jurisdiction **is not a retry. It is a new decision**, and it is re-evaluated against the constraint set from the top.

Rail execution

> Do not force authorize, capture and reverse onto every rail. Six of the seven shapes do not have all three.

RAIL	SHAPE	THE PROPERTY THAT BREAKS CARD ASSUMPTIONS
Card	Intent → authorization → capture → clearing → settlement	Reversible before capture, disputable after
ACH	Intent → submission → acceptance → return window → settlement	No authorization at all, and a return window that can run to sixty days — which removes short-horizon promotion on this rail entirely
RTP / FedNow	Intent → authentication → push → final settlement	**Irreversible on receipt.** There is no reversal to issue, so recovery becomes a return request the receiver may decline
Open banking	Consent → authentication → instruction → settlement	Authentication precedes and authorises the payment rather than sitting inside it
Wire	Intent → instruction → pending → settlement	Recall is manual and negotiated
AFT / OCT	Funding instruction → network processing → settlement	Push to card. No capture, different dispute rights
Wallet	Intent → wallet authorisation → underlying rail	Inherits the shape of whatever funds it

The common abstraction is the payment intent (rail-agnostic and unchanged), a payment request that gains a rail type and rail operation, a rail execution with its own state machine, a rail outcome normalised into the canonical vocabulary, and a money movement that is rail-aware.

SEVEN THINGS A RAIL ADAPTER MUST DECLARE BEFORE IT CAN BE ENABLED

`states[]` · `reversibility` (reversible until a named state, or irreversible) · `finality_point` · `return_window` · `authentication_position` (inside, preceding, or none) · `recovery_actions[]` · `dispute_rights`.

A rail that has not declared these cannot be turned on. The consequences run through attempt strategy, recovery, authentication, money movement and learning: **retry is meaningless past a finality point**, and compensation is rail-declared rather than assumed.

Recovery

> The unowned domain. An authorization is dispatched, no response arrives within the deadline, and it may have approved, declined, or never arrived at all.

The orchestrator cannot decide, because the answer depends on the adapter's declared timeout disposition, the network's stand-in behaviour, the message model and the merchant's exposure tolerance. The module cannot decide, because it is stateless and already gone. Therefore a named supervisor owns it.

TRIGGER	RESOLUTION PATH	DEADLINE
Authorization timeout, unknown outcome	Reversal advice, then status inquiry, then reconciliation against network clearing	Immediate → 24 h → settlement window
Approved authorization, capture failed	Retry capture within the window; if impossible, reversal	Capture window
Partial success in a multi-leg order	Order-level compensation policy: hold, refund collected legs, or continue	Order expiry
Authorization succeeded, downstream state write failed	Journal reconciliation; state rebuilt from events	Seconds
Orphan authorization found at clearing	Attach to the intent if identifiable, otherwise an exception case	Settlement
Duplicate authorization detected	Reverse the later one; alert if it was captured	Immediate
Step-up completed but the original request expired	A new request under the same intent, carrying the authentication evidence	Authentication validity window

Compensation is declared alongside the execution graph, versioned and testable — on timeout, on unknown, on late approval after abandonment, on capture failure, and on order-leg failure via a merchant-configurable policy reference.

THE METRIC IS UNRESOLVED-OUTCOME AGE, NOT INCIDENT COUNT

Recovery is a domain with an on-call owner, an error budget and a reconciliation SLA measured against the settlement window. Counting incidents rewards closing cases; measuring the age of the oldest unresolved outcome rewards resolving them.

Tenant isolation extends here too, and this is the quota usually missed: **one tenant's incident must not starve another tenant's recovery capacity.** [A]

PART

III — Credential, security and money

10 Credential and instrument

11 Platform security

12 Money movement

13 Cross-border, FX and multi-currency

Credential and instrument

> The vault exposes references. Credentials never enter application logs, never appear in module inputs, and never cross a service boundary except to an adapter inside the PCI boundary.

A credential carries its id — the only thing modules ever see — plus scope, type, network, token requestor, token reference, lifecycle state, last four, expiry, credential age, updater status, network token status, and the enriched attributes. Token types are `FPAN · DPAN · MPAN · acquirer_token · psp_token`.

Vault-time enrichment

This is described in the source as the single highest-leverage latency decision in the architecture, and the description is right.

```
VAULT TIME                                  asynchronous, unbounded budget
  credential stored
    → scheme range file lookup (Visa, Mastercard, Discover)
    → US debit overlay resolution
    → PINless eligibility resolver  (a versioned pure function)
    → attributes written alongside the credential reference

AUTHORIZATION TIME
  credential_id → attributes already present        0 ms, in memory
```

The eligibility resolver is a versioned pure function over the range snapshot and the merchant's debit configuration. It is reproducible eighteen months later during a network audit, and it is not permitted to consult live state. When the snapshot moves, a re-enrichment sweep runs over the affected ranges, versioned, with the before-and-after diff retained.

Freshness, and five invalidation channels

CLASS	ATTRIBUTES	REFRESH
Volatile	Debit network participation, PINless eligibility, regulated status	On every network file delta affecting the range; verify on first use after 24 hours
Semi-stable	Product id, commercial flag, prepaid flag	Weekly sweep, plus on delta
Stable	Issuer country, network	On delta only

Invalidation arrives through a scheduled sweep, a network file delta, portfolio and range change events, first-use verification — and, fourth, **observed contradiction**: a network response that disagrees with a stored attribute invalidates it and schedules re-enrichment. That is the channel that catches what the other three miss.

WHY THERE IS NO ON-PATH VALIDATION CALL

A reviewed design proposed real-time cache validation inside context assembly at a one-millisecond budget. It was not adopted, for a reason worth keeping: **a validation that must return in one millisecond is a lookup that times out under load and falls back to the cached value anyway** — latency paid, nothing gained.

A fixed seven-day verification window was declined on a different ground: debit network participation and issuer country do not decay at the same rate, so one window is wrong for both. Hence freshness classes, and staleness as a decision input rather than an error. [A]

Lifecycle, and the Forward API

Credential lifecycle runs `CREATED → ACTIVE`, then through updates (which produce a new attribute version and retain the prior one), suspension, expiry, replacement, revocation, and `COMPROMISED` as a terminal state that triggers merchant notification. The network token lifecycle is tracked separately and joined by reference: `PROVISIONING · ACTIVE · SUSPENDED · RESUMED · DELETED · UPDATED`.

THE ARCHITECTURAL CONSEQUENCE OF PORTABILITY

The vault's competitive position rests on credentials being portable rather than captive. The Forward API lets an enterprise merchant execute against a credential held in our vault through a processor that is not us.

The consequence is structural and easy to lose: **the vault must be usable without the orchestrator.** Credential references are stable, independently authorised objects, not handles that only mean something inside our execution path. A vault that requires our orchestrator to be useful is not portable, whatever the endpoint is called. [A]

Note that CI-001 records this capability as matched by a competitor. That changes how it is sold, not whether it is built — see CI-001 section 13.

Platform security

SEVEN CRYPTOGRAPHIC ZONES · AND THE ONE RULE THAT HOLDS THEM

ZONE 0	INTERNET	TLS termination, DDoS, bot defence	non-CDE
ZONE 1	API EDGE	authn, idempotency, schema	non-CDE
ZONE 2	PAN INGRESS	the only place a raw PAN may enter	CDE entry
ZONE 3	CREDENTIAL VAULT	tokenisation, storage, provisioning	CDE
ZONE 4	HSM / KEY	keys, P2PE, PIN translation — no application code	CDE, highest
ZONE 5	PROCESSOR ADAPTER	detokenisation for the network message only	CDE egress
ZONE 6	NETWORK		external

EVERYTHING ELSE OPERATES ON CREDENTIAL REFERENCES

Orchestrator, decisioning, learning, experimentation, money movement, reporting. One module taking a PAN as input collapses the whole boundary.

— Replay is its own zone with equivalent controls. An engine that can re-execute any historical transaction is an oracle over payment behaviour, and treating it as an operations feature is the likeliest route to compromise.

Keys, identity and access

CONTROL	SPECIFICATION
Key hierarchy	Four tiers: a root of trust (HSM-resident, dual-control ceremony, offline), zone master keys per environment and region, data encryption keys per credential class, and working keys rotated on schedule and on event.
Rotation	Scheduled per key class; event-driven on personnel change, suspected compromise or failed attestation. ***Rotation is exercised, not merely documented — an unexercised rotation is an untested rollback.***
Cryptographic erasure	A credential's destruction is the destruction of its key material, not a delete of a row.
Service identity	Workload identity issued at deploy, short-lived, non-transferable. mTLS between every service. **No network-position trust.**
Service authorization	Explicit per-call policy, default deny.
Operator access	Named, time-bound, purpose-recorded. **No standing production access to CDE zones.**
Break-glass	Dual control, pre-authorised scope, full session recording, mandatory post-review within 24 hours.
Secrets	Brokered at runtime, never in images, config bundles or plan artefacts. A signed plan bundle carries no secret.
Log redaction	Enforced at serialisation, not by convention.

REPLAY IS A SECURITY BOUNDARY, NOT A TOOL

This was the strongest single point in the investment review, and it is easy to underrate. An engine that can re-execute any historical transaction is **an oracle over historical payment behaviour**. Treating it as an operations feature would be the most likely route to compromise.

So replay runs as its own zone with CDE-equivalent controls. Credential material never leaves it. Every replay is attributed to a named operator, and replay of another operator's traffic requires the same dual control as break-glass. Replay may never dispatch to a processor. [A]

Tenant separation is a security property rather than a performance one. Cross-tenant exposure in a platform serving PayFacs and submerchants is a breach, not a bug — so authorization is evaluated against the client hierarchy on every access, including internal service calls, and including the replay and analytics paths.

Screening, which sits here rather than in decisioning

CHECK	PLACEMENT	BUDGET	ON FAILURE
Sanctions — party screening	Onboarding and periodic rescreen	Off path	Account block
Sanctions — transaction screening	Auth path, parallel node	≤ 12 ms p99	Hard fail, no fallback
Cross-border and country risk	In the constraint set	0 ms, precomputed	Constraint block
AML transaction monitoring	Post-authorization, event-driven	Off path	Case creation
Prohibited MCC or activity	In the constraint set	0 ms, precomputed	Constraint block

Screening asks whether a party is on a list; policy asks whether this transaction is permitted. Both hard-fail. Screening is deliberately opaque to the merchant; policy is explained, with the authority level named. The screening ***result*** remains available as a feature to other decisions — it is the decision that is not experimentable, not the data.

Money movement



Ledger accounts and entry rules

Eleven account types: `merchant_receivable · merchant_reserve · scheme_receivable · interchange_expense · scheme_fee_expense · processing_revenue · fbo_clearing · dispute_liability · refund_liability · chargeback_reserve · platform_payable`.

Every entry references an event, and optionally an obligation and an attempt. The ledger is derived from the journal and never written to directly. Reconciliation runs daily against network settlement files, and **breaks produce cases, not alerts**.

Three classes of financial invariant, evaluated continuously

CLASS	INVARIANTS
Sequence	Σ authorized ≥ Σ captured ≥ Σ presented ≥ Σ settled · Σ refunded ≤ Σ settled · every capture traces to an authorization within its window
Balance	opening + credits − debits = closing, per account, per currency, per legal entity, per day · network settlement = ledger settlement = merchant payable + fees + reserves + adjustments · no cross-currency netting inside an entity position
Completeness	Every monetary event has a journal event · every journal event with monetary meaning has balanced ledger entries · no orphan authorization at clearing beyond the resolution SLO

THREE DATES, NOT ONE

Every monetary event carries an **economic date**, a **processing date** and a **value date**, alongside currency, legal entity, account, source, destination, amount, exchange rate and its source event.

Collapsing those three is named in the source as the most common cause of unexplainable reconciliation breaks, and the claim matches experience. A related rule: the day closes at **UTC midnight, universally** — not merchant local time, not the acquirer settlement cut-off. Those are separate fields, reconciled against, never the boundary. [A]

Disputes, funding and pricing

A dispute carries its classification, reason code and network; its liability, which the authentication evidence determines; the evidence lifecycle through representment, pre-arbitration and arbitration; an outcome of `WON | LOST | ACCEPTED | EXPIRED`; and the financial adjustment and reserve impact, linked back to the attempt, intent and obligation. **Dispute probability by routing choice, credential type and authentication outcome becomes a decision feature** — which is the difference between processing chargebacks and reducing them.

Funding covers payout timing per merchant, reserves (rolling, fixed or dynamic, held in the ledger and visible to the merchant with a reason), FBO structures for platform and PayFac models, negative balance as an explicit state with a collection path, and split settlement native via obligation discharge rather than post-hoc transfers.

Pricing consumes interchange (versioned, bitemporal), scheme fees, processor cost and the margin model. Two requirements do the work: **routing consumes the same versioned cost tables as billing**, or the merchant statement will not reconcile to the decisions; and a merchant statement must be reconcilable to per-attempt decisions.

Cross-border, FX and multi-currency

> There are four currency decisions, made in four places, with four different consequences. Treating FX as a routing attribute is the mistake this section exists to prevent.

DECISION	MADE WHERE	CONSEQUENCE
Presentment — what the cardholder is charged in	Merchant, at intent creation	Conversion economics and disclosure obligations
Processing — what the authorization is submitted in	Routing, constrained	Acquirer eligibility, scheme cross-border fees
Settlement — what we receive from the network in	The acquiring relationship	Exposure window and conversion cost
Payout — what the merchant receives	Funding configuration	Merchant FX cost and local account requirements

The rate lock binds to the intent, not the attempt

Every attempt on an intent uses that intent's lock. A retry after expiry requires a requote, and where a rate was shown, a fresh disclosure. Both the locked rate and the realised rate are recorded and the difference is booked as an FX result. On expiry the lock resolves to requote, auto-extend at risk, or fail — declared, not defaulted.

A RATE LOCK IS A LIABILITY FROM THE MOMENT IT IS DISCLOSED

It appears in the exposure position from disclosure, not from settlement. The multi-currency position therefore carries ten fields per currency per legal entity, including disclosed-but-unsettled locks, open FX exposure, hedge position, and realised and unrealised result.

Every conversion is a ledger event with its rate, source, execution time and counterparty — never an implicit revaluation. The platform holds and reports the position; treasury sets policy. **Where that boundary sits is explicitly unsettled**, and it is open item D10 in DOC-000: either treasury inherits a position it cannot control, or the platform starts making hedging decisions it should not. [A]

DCC, and what it is never allowed to influence

Eligibility, a genuine offer in both currencies with the margin disclosed, and the choice recorded with timestamp and presentation evidence retained for the dispute window. Absent evidence of a free choice, DCC is not applied.

Never permitted: defaulting the cardholder into DCC; applying it where they declined; and **routing selection influenced by DCC revenue** — DCC margin is excluded from the routing objective function entirely. That exclusion was adopted without a recorded reason while the rest of the same argument was accepted, and it is open item D11.

Local acquiring buys higher approval rates and domestic interchange, and costs a local licence or partner and a local settlement account. Cross-border acquiring is simpler operationally and materially worse on approval in several markets. Which markets we acquire in is **entitlement-plane state, not routing configuration** — and the build-partner-defer decision per market is open item D1, the one most likely to invalidate the commercial case rather than merely delay it.

PART

IV — Records, interfaces and getting there

14 State machines

15 Journal, determinism and replay

16 Canonical results and disclosure

17 The API surface

18 Latency and data placement

19 Migration and legacy coexistence

20 What this document does not answer

State machines

> The internal vocabulary is large because operations needs precision. The external vocabulary is small because merchants need stability. Never export the internal set.

ENTITY	STATES
Payment intent — external	`requires_action` · `processing` · `authorized` · `partially_captured` · `captured` · `failed` · `canceled` · `partially_refunded` · `refunded` · `disputed` (ten)
Internal, never surfaced	`AUTHORIZATION_DISPATCHED` · `AUTHORIZATION_UNKNOWN` · `PROCESSOR_TIMEOUT` · `NETWORK_TIMEOUT` · `STAND_IN_APPROVED` · `REVERSAL_PENDING` · `REVERSAL_CONFIRMED` · `REVERSAL_FAILED` · `STEP_UP_PENDING` · `RECOVERY_HELD` · `SUPERVISOR_CASE_OPEN` · `CLEARING_MISMATCH` · `ORPHAN_AUTHORIZATION` · `COMPENSATION_IN_PROGRESS` · `LEDGER_BREAK` (fifteen)
Order	Created, collecting, partially collected, fully collected, discharging, settled — with a compensation branch and terminal failed, expired or canceled
Obligation	Open, partially discharged, discharged, disputed, written off, cancelled
Credential	Created, active, active-updated, suspended, expired, replaced, revoked, compromised (terminal)
Network token	Provisioning, active, suspended, resumed, deleted, updated
Authentication	Requested → evaluating → one of: not required, exemption claimed (accepted or soft-declined to step-up), delegated, or 3DS requested → frictionless, challenge or unavailable → complete
Money movement	Authorized, captured, presented, cleared, settled, funded, reversed
Rail execution	**Rail-specific, declared by the adapter, never assumed**

THE AUTHENTICATION BOUNDARY RULE

Authentication context binds to the **payment intent**, not the payment request. That is what makes step-up-then-retry and processor substitution work without re-challenging the cardholder: a completed authentication carries its evidence bundle forward to every subsequent attempt on the same intent.

Bind it to the request instead and every recovery path re-challenges, which converts a recovered sale into an abandoned one. [A]

Journal, determinism and replay

> Current state plus complete event history. State is a projection; the journal is the record, and its schema is versioned and governed as a public contract.

Thirty-four event types are journalled, from `ObligationCreated` through context assembly and plan resolution, every decision (`ScreeningCompleted`, `ConstraintsBound`, `RiskEvaluated`, `AuthenticationDecided`, `CredentialSelected`, `MessageEnriched`, `RouteSelected`, `StrategyDecided`), the attempt lifecycle including `AttemptOutcomeUnknown`, the money events, and recovery and dispute resolution.

The determinism snapshot

Bound at plan resolution and stored on the request, so that “why did this payment behave differently yesterday?” has an answer rather than a theory.

```
{ exec_id, plan_version,
  modules   { screening, constraint, risk, authentication,
              credential, routing, strategy, adapter },
  artefacts { policy_set, routing_table, bin_snapshot,
              cost_table, risk_model, route_model },
  bitemporal_coordinates { transaction_time, control_plane_as_of },
  experiment_assignment { ... } }
```

REPLAY MODE	WHAT IT DOES	GUARDRAIL
Shadow	Re-executes against candidate versions	**Never dispatches to a processor.** Not configurable.
Diff	Original versus candidate decisions, per node	Runs inside the PCI boundary with production masking rules
Counterfactual	Estimates outcome under a candidate policy, off-policy	**Explicitly an estimate with a confidence interval, never presented as a fact.** Where the logs hold no overlap, the answer is “insufficient evidence”

A SIMULATOR THAT ALWAYS ANSWERS IS A SIMULATOR THAT IS SOMETIMES LYING

Counterfactual estimates require overlap between logged behaviour and the proposed alternative. Where a merchant has never had traffic on the alternative, the honest output is a refusal, not a confident number derived from a different population.

This is why the merchant digital twin is deferred rather than built: without overlap a twin does not estimate, it extrapolates — and it will do so most confidently for the merchants with the least data. [J]

Canonical results and disclosure

Processor codes never reach merchants. Fourteen canonical codes, four classes, and a retryability block that the attempt strategy consumes while merchant support consumes the action fields.

FIELD	VALUES OR PURPOSE
`code`	`CARD_DECLINED · INSUFFICIENT_FUNDS · INVALID_CARD · EXPIRED_CARD · AUTHENTICATION_REQUIRED · AUTHENTICATION_FAILED · POLICY_BLOCKED · SCREENING_BLOCKED · RISK_BLOCKED · PROCESSOR_UNAVAILABLE · NETWORK_TIMEOUT · DUPLICATE · INVALID_REQUEST · UNKNOWN`
`class`	`SOFT | HARD | TECHNICAL | UNKNOWN` — this is what attempt strategy reads
`retryability`	Retryable flag, permitted actions, earliest retry time
`customer_action` / `merchant_action`	What each party must do, if anything — this is what support reads
`liability`	Who bears loss if this proceeds
`raw`	Processor code, network code, ISO field, processor message — restricted

Four tiers of disclosure

DATA	MERCHANT	PLATFORM / PAYFAC	OPERATIONS	ENGINEERING
Status, amount, last four, network	Full	Full	Full	Full
Canonical decline class and actions	Full	Full	Full	Full
Authentication result and liability	Full	Full	Full	Full
Attempt sequence and strategy origin	Summary	Full	Full	Full
Processor and acquirer identity	Contractual	Contractual	Full	Full
Route decision and reason	Reason only	Reason only	Full	Full
Risk score	Band	Band	Full	Full
Constraint decision and authority level	Reason only	Reason + own rules	Full	Full
Exec id and plan version	Reference only	Reference only	Full	Full
Module versions and dependencies	No	No	Full	Full
Experiment assignment	On request	On request	Full	Full
Raw network message	No	No	Restricted	Restricted
Credentials	**Never**	**Never**	**Never**	**Never**

DELEGATION OVERRIDES THE TABLE, AND THE "WHY NOT" VIEW COSTS NOTHING

These are constraint-plane questions resolved per request and logged, not a static access matrix. What a PayFac may see about its submerchant is a policy question with an answer that can change.

Every decision logs its considered action set and a canonical reason per rejection — `BIN not supported for this acquiring entity`, `higher expected dispute cost for cohort`, `health degraded at decision time`, `would breach concentration bound`. The “why not” view **costs nothing to produce because the considered set is already logged**, and it is disproportionately valuable to sophisticated merchants. Reason codes are canonical and countable, never free text.

The API surface

TIER	ENDPOINTS	WHO REACHES IT
Tier 1 — the path almost everyone takes	`POST /v1/payments` (create and confirm in one call) · `POST /v1/payment_intents` · `/capture` · `/refund` · `POST /v1/payment_methods`	Standard checkout. **No field is required in tier 1 that a standard card checkout does not already have.**
Tier 2 — reached only when the commercial structure needs it	`POST /v1/obligations` · `POST /v1/orders` · `POST /v1/payment_requests`	Marketplaces, franchises, instalments, B2B

Alongside those: customers, authentication sessions, credentials (including `POST /v1/credentials/{id}/forward`), transfers, settlements, payouts and disputes, plus attempt and event listings per intent. An idempotency key is required on every mutating call.

The internal surface

Timeline, execution, attempt trace, attempt decisions, the restricted message view, the determinism snapshot, recovery cases, constraint resolution (“what applies, and why”), plan retrieval by version, and the mutating operations — replay (shadow only), retry, reverse, and recovery case resolution. All under RBAC, with dual control on mutating operations, full audit, and credential masking enforced at serialisation.

THREE MANDATORY FIELDS IN THE DECISION CONTRACT

Every decision point answers in the same shape, and three fields are enforced by schema validation at registration: `propensity` (1.0 for deterministic strategies — populate it anyway), `fallback_would_have_been`, and `rejected[]` with canonical reason codes.

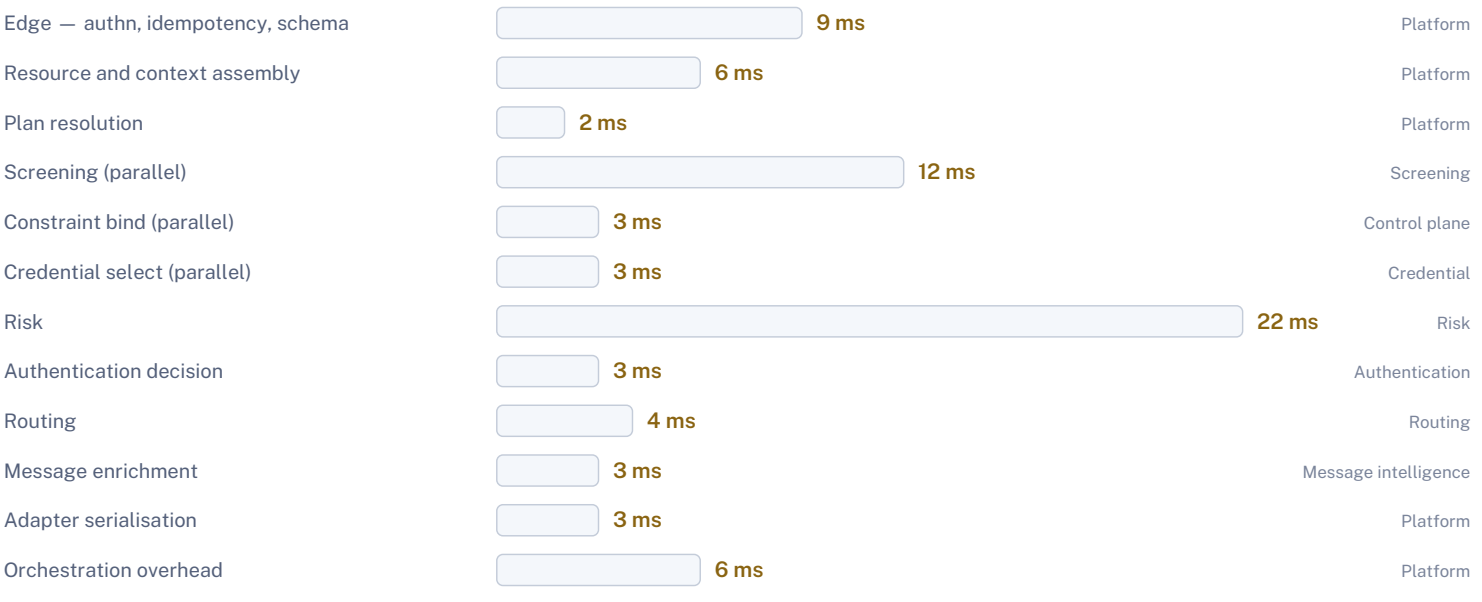
A decision point that cannot populate all three fails validation and cannot be registered. Neither propensity nor the fallback can be backfilled for traffic that has already run, which is why logging them should begin on the current engine before this platform exists. That is the single most time-sensitive item in the whole design. [A]

The event contract

At-least-once delivery, so duplicates are possible by design; a stable idempotent event id; sequence per payment intent, monotonic, with **no global ordering offered**; every payload signed with key rotation published in advance. Retry backs off exponentially to a bounded horizon and then dead-letters, with delivery state queryable per event, and replay of a delivered or failed event uses **the original payload, never a regenerated one**.

What we do not do: guarantee ordering across intents, guarantee exactly-once, or hide schema versions. And the sandbox must produce every canonical result on demand — including timeout with unknown outcome, late approval after abandonment, step-up mid-flow and partial capture. A sandbox yielding only approvals and declines produces integrations that break on the first unknown-outcome timeout.

Latency and data placement



— Each line has a named owner, and a module over its budget is treated as a product regression rather than an observability metric. The earlier 40 ms and 300 ms figures are both retired; neither was defended.

THREE RETIRED NUMBERS

Under 40 ms was proposed with a per-stage budget and declined: not achievable with mandated controls on the path. **Under 300 ms** appeared in the same document's closing ask — a 7.5× spread inside one document — and was still being quoted as a binding constraint two rounds after the architecture had moved. **A 150 ms rollback tolerance** was rejected as two and a half times the entire added-latency budget.

The live commitment is **platform-added latency under 60 ms at p99**, with screening included and risk on the path. The end-to-end figure is not a platform commitment, because the processor, network and issuer dominate it and we do not control them. [A]

Three data tiers, and one rule

TIER	CONTENTS	REFRESH
Hot — in process, or one hop, never a query	Plan bundles, constraint sets, routing tables, cost tables, merchant configuration, sanctions lists, model artefacts, and instrument attributes carried with the credential reference	Push: versioned, signed, with rollback
Warm — asynchronous, off the path	Event journal (append-only, partitioned), transactional store for resource state, decision records for learning, operations search index	Stream
Cold	Lakehouse, settlement files, model training sets, archives	Batch

> Nothing on the authorization path issues a query whose latency we do not control. If a module needs data it does not have, the answer is a push channel, not a lookup.

Retention follows the obligation rather than convenience: credentials per PCI scope and in the vault only; authentication evidence per the scheme liability window; raw network messages short, restricted and encrypted; decision records long, because off-policy evaluation and model validation need them; the journal long, for dispute and audit. Residency zone is a context attribute, and it **constrains where a plan may execute**.

Migration and legacy coexistence

> The failure this exists to prevent is split-brain: two systems that each believe they hold the truth about a transaction, diverging silently, discovered at settlement.

FIVE CUTOVER STAGES · SINGLE WRITER PER RESOURCE CLASS THROUGHOUT

SHADOW	v3 executes alongside legacy, dispatches nothing, decisions diffed
MIRROR	v3 owns reads; legacy still owns writes; divergence measured continuously
CUTOVER	the ownership marker flips for the slice; legacy becomes a projection consumer
DRAIN	in-flight legacy-owned transactions finish under legacy ownership
RETIRE	the legacy path is disabled for the slice

ROLLBACK IS AN OWNERSHIP-MARKER FLIP PLUS A DRAIN, AVAILABLE AT EVERY STAGE UP TO RETIRE

Ownership never changes for a transaction already in flight. Dual-write is rejected outright: there is no atomicity across two stores, so it produces the divergence it exists to prevent.

— The shadow gate is two-class: invariants at zero, always; decisions explained rather than identical. A zero-discrepancy decision bar either never opens or is met by tuning the new engine to imitate the old one, which discards the reason for building it.

Single writer per resource class

RESOURCE	PHASE 1	PHASE 2	PHASE 3
Credentials	v3	v3	v3
Payment intents	legacy	v3	v3
Attempts	legacy	legacy	v3
Clearing and settlement	legacy	legacy	v3
Ledger	legacy	legacy	v3

Every resource carries its owning system, the time it was owned since, and a cutover id. **A write to a resource this system does not own is rejected** — not queued, not merged, not reconciled later. Propagation is one-way: a transactional outbox in the same transaction as the state change, through change data capture and a translation layer, into the journal.

DUAL-WRITE IS REJECTED OUTRIGHT, AND SO IS THE ZERO-DISCREPANCY GATE

Dual-write has no atomicity across two stores, so its failure mode is precisely the divergence the pattern exists to prevent, appearing under load. A global write-path flip with legacy as a read-only archive was also declined: it discards the per-resource-class ownership that makes rollback possible.

And the gate: **zero shadow discrepancies over fourteen days was rejected because the new engine is meant to decide differently.** A zero bar either never opens, or is met by tuning the new engine to imitate the old one — which discards the reason for building it. Replaced with two classes: invariants at zero always, and decision divergence *explained* by category rather than eliminated. [A]

The ramp ladder

STEP	POPULATION	MINIMUM SOAK	ADVANCE REQUIRES
0	Synthetic and internal test accounts	Continuous	Invariants at zero
1	Our own merchant traffic	7 days	Invariants at zero; the unexplained divergence bucket empty

2	Opt-in mid-market: low ticket, single currency, domestic	14 days	Above, plus acceptance rate non-inferior
3	Opt-in mid-market, full ticket range	14 days	Above, plus platform-added latency within budget
4	General mid-market, non-opt-in	30 days	Above, plus a full settlement cycle reconciled
5	Enterprise, by named account, with the account informed	Per account	Above, plus account-level agreement

Cross-border, multi-currency and card-present are separate ladders rather than steps within this one. Strategic accounts are never in an early step, and never in the same step as the first exercise of a capability.

Rollback triggers

TRIGGER	THRESHOLD	ACTION
Acceptance rate regression	Below the matched legacy baseline beyond the interval, sustained across two intervals at minimum volume	Halt ramp, then revert the slice
Write-path error rate	5xx above 0.05% of slice volume over a rolling 3 minutes, minimum 2,000 transactions	Immediate revert
Platform-added latency	p99 above budget by more than 20 ms for two consecutive 5-minute intervals	Halt ramp; revert if sustained
Unknown-outcome rate	Above the recovery SLO	Immediate revert
Ledger or reconciliation divergence	Any non-zero invariant breach	**Freeze ownership, drain, supervisor case** — then revert once the position is understood. Not an immediate revert
Emergency constraint propagation failure	Any node failing to confirm within deadline	Slice stops accepting traffic, fail-closed

THE POINT OF NO RETURN, NAMED PER RESOURCE CLASS

Payment intents: the first intent with multiple obligations. Attempts: the first step-up-recovered attempt. Ledger: the first multi-currency position. Obligations: the first obligation outliving its order. After each boundary, forward-fix is the only path.

This was not raised by any review. It is here because the boundary exists whether or not anyone writes it down, and the cost of discovering it during a rollback is not recoverable. [A]

What ships first

Credential vault and Forward API on new writes only → the resource model as a read-through facade → instrument enrichment feeding legacy routing first → journal ingestion via CDC, so observability precedes control → **decision logging on the legacy stack** → control plane in advisory mode → execution engine in shadow → cutover by slice, narrowest first.

The fifth item is the one that does not look urgent and is: propensity, considered set and fallback emitted from the current routing and retry decisions, with no behaviour change, builds the evaluable history that everything downstream needs and that cannot be reconstructed later.

What this document does not answer

> Eighteen items, renumbered into sequence. All are carried into the consolidated register in DOC-000 section 06, which is where they should be tracked.

#	GAP	WHY IT MATTERS
1	Settlement and funding are shallower than the authorization path	The domain with the most regulatory weight has the least design.
2	Dispute lifecycle is sketched, not designed	Representment and network dispute rails will be built ad hoc otherwise.
3	Platform FX position versus treasury hedging authority	Either treasury inherits a position it cannot control, or the platform makes hedging decisions it should not.
4	DCC as revenue versus DCC as a cardholder service	Excluded from routing; the pricing posture is unsettled.
5	**Local acquiring footprint per market**	Determines whether cross-border approval parity is achievable at all. **The one to watch** — it can invalidate the commercial case rather than delay it.
6	Whether learned strategy can be defended to enterprise procurement	If not, higher autonomy is internal-only and the volume advantage shrinks.
7	Data residency versus centralised learning	May force per-region models with materially less data each.
8	Obligation's boundary against the enterprise general ledger	Risk of building a second general ledger by accident.
9	Non-card rails have semantics this design assumes away	Recovery and attempt strategy are card-shaped. The domain exists; the rails do not.
10	Terminal and card-present estate integration	Context assembly assumes richer terminal data than legacy estates provide.
11	Organisational feasibility of the domain count	Under-staffed domains become shared-service bottlenecks and recreate the monolith socially. See ST-001.
12	Whether merchant experiment opt-out materially degrades learning	If the largest merchants opt out, learning happens on the tail, where evidence is thinnest.
13	Individual rail designs	ACH returns, RTP return requests, open banking consent lifecycle. Each needs its own pass before that rail carries volume.
14	Who owns the bounds on the approval-versus-cost exchange rate	The objective function is commercial policy wearing a model artefact's clothing. Unowned, the optimizer settles it.
15	Strategy contention under client self-serve authoring	Tier three authoring should not be promised until this is answered.
16	Who allocates the platform-funded exploration pool	Exploration is a finite resource denominated in lost approvals, and an unowned budget is spent by whoever asks first.
17	Card present — build a device platform, partner, or restrict to the existing estate	A multi-year lead time and a certification calendar that does not compress.
18	Population overlap detection over arbitrary predicates	The controlled segment vocabulary caps concurrency; whether that cap binds before a general solution exists is unknown.

AND THREE THAT ARE NOT ON THAT LIST

The absent concepts from section 01 — PAR, vaultless tokenisation, and key epochs — are not gaps the source acknowledged. They are gaps consolidation found. PAR in particular is load-bearing for the omnichannel case, because it is the only cross-channel link between a wallet token and its funding account, and nothing in this design provides that link. [J]